

地铁跑酷强化学习项目文档

DeepSeek

2026 年 2 月 19 日

摘要

本文档详细介绍了基于地铁跑酷（Subway Surfers）游戏的强化学习项目。项目实现了自定义的 Gymnasium 环境，通过屏幕捕获、OCR 识别、鼠标模拟等技术实现与真实游戏的交互。支持在线 PPO 训练和离线 CQL 训练，并包含自动重启、防卡死等鲁棒性机制，确保长时间无人值守训练。

1 项目简介

本项目旨在将强化学习算法应用于经典跑酷游戏《地铁跑酷》。由于游戏本身是闭源的 Windows 可执行文件，我们通过计算机视觉和输入模拟技术将其封装为标准的 Gymnasium 环境，从而允许强化学习智能体进行训练。项目规划分为三个阶段：

1. 环境封装与功能测试（鼠标控制、OCR 信息提取、自动重启、防卡死）。
2. 在线训练（PPO）收集初步数据并验证环境。
3. 离线训练（CQL）基于收集的数据集进行高效学习。

2 环境搭建与依赖

运行本项目需要以下 Python 库：

```
1 pip install gymnasium opencv-python pyautogui mss numpy torch torchvision stable-  
   baselines3 d3rlpy easyocr pynput
```

此外，需确保游戏可执行文件路径正确，并校准屏幕捕获区域和按钮坐标。

3 环境类 SubwaySurfersEnv 详解

环境类继承自 `gym.Env`，核心功能包括屏幕捕获、OCR 数字识别、鼠标滑动模拟、自动重启以及防卡死机制。

3.1 初始化参数

```
1 def __init__(self, render_mode=None, game_exe_path=r"...",  
2             stuck_threshold=10, enable_stuck_detection=True):
```

- `render_mode`: 渲染模式（human 或 rgb_array）。

- `game_exe_path`: 游戏可执行文件路径。
- `stuck_threshold`: 连续多少步里程数不变判定为卡死 (默认 10)。
- `enable_stuck_detection`: 是否启用防卡死机制。

3.2 观察空间与动作空间

- 动作空间: 离散 5 维, 0-无操作, 1-左, 2-右, 3-上, 4-下。
- 观察空间: (120,160,3) 的 RGB 图像, 像素值 0-255。

3.3 图像获取与 OCR 识别

使用 `mss` 快速截屏, `OpenCV` 预处理, `easyocr` 识别数字区域 (里程数、金币数、死亡分数)。

```

1 def _ocr_digit(self, region):
2     img = np.array(self.sct.grab(region))
3     gray = cv2.cvtColor(img, cv2.COLOR_BGRA2GRAY)
4     _, binary = cv2.threshold(gray, 200, 255, cv2.THRESH_BINARY)
5     result = self.ocr_reader.readtext(binary, allowlist='0123456789')
6     if result:
7         text = result[0][1]
8         digits = ''.join(filter(str.isdigit, text))
9         return int(digits) if digits else 0
10    return 0

```

3.4 奖励函数

每步存活奖励 0.1, 里程数每增加 1 奖励 0.01, 金币每获得 1 个奖励 0.1, 死亡惩罚 -10。

```

1 def _compute_reward(self, info):
2     reward = 0.1
3     mileage_gain = info["mileage"] - self.last_mileage
4     if mileage_gain > 0: reward += mileage_gain * 0.01
5     coin_gain = info["coin"] - self.last_coin
6     if coin_gain > 0: reward += coin_gain * 0.1
7     if info["collision"]: reward -= 10
8     return reward

```

3.5 动作执行

通过 `pyautogui` 模拟鼠标滑动, 每次滑动前将鼠标移至窗口中心。

```

1 if action != 0:
2     pyautogui.moveTo(self.center_x, self.center_y)
3 if action == 1:
4     pyautogui.dragRel(-self.swipe_offset, 0, duration=self.swipe_duration)
5 # ... 其他方向

```

3.6 自动重启逻辑

环境支持全自动启动和死亡后重启。

- **首次启动**：若进程不存在，启动游戏并依次点击主菜单 PLAY 按钮（坐标 (1418,1074)）和开始按钮 (1155,1066)。
- **正常死亡**：检测到死亡画面后，点击死亡 PLAY 按钮 (1401,1123) 重启。
- **卡死强制重启**：若卡死超时，杀死进程重新启动（详见防卡死机制）。

3.7 防卡死机制

针对游戏可能出现的“穿模卡死”现象，设计了以下检测与处理流程：

1. 每步记录当前里程数，若连续 `stuck_threshold` 步未变化，进入“卡死模式”。
2. 卡死模式下强制将动作置为 0（无操作），并累计卡死步数。
3. 若卡死模式持续超过 `stuck_terminate_threshold`（默认 `stuck_threshold*3`），则强制结束本回合，并标记结束原因为“stuck”。
4. 下一次 `reset` 时，若原因为“stuck”，则杀死当前游戏进程并重新启动，确保完全恢复。

4 数据收集

为了进行离线训练，需要收集大量轨迹数据。项目提供两种收集方式。

4.1 随机策略数据收集

`collect_random.py` 使用随机动作与环境交互，每 `save_interval` 步保存一次 pickle 文件。

```
1 def collect_random(total_steps=80000, save_interval=10000):
2     # 循环内随机采样动作，记录 (obs, action, reward, next_obs, done)
```

4.2 人工演示数据收集

`manual_demo.py` 通过键盘方向键控制角色，实现高质量轨迹录制。按键映射为离散动作，每次按键触发一次滑动，松开即恢复无操作。

```
1 class ManualRecorder:
2     def on_press(self, key):
3         if key == keyboard.Key.left: self.current_action = 1
4         # ... 其他方向
5         self.need_action = True    # 触发式执行
```

收集的数据同样保存为 pickle 格式，便于后续合并。

5 在线训练 (PPO)

使用 Stable-Baselines3 的 PPO 算法进行在线训练，旨在快速验证环境并收集初步数据。

```
1 from stable_baselines3 import PPO
2 model = PPO('CnnPolicy', env, verbose=1, tensorboard_log='./logs/')
3 model.learn(total_timesteps=1000000)
```

训练过程中每 10 万步保存一次模型，并可通过 TensorBoard 监控学习曲线。

6 离线训练 (CQL)

基于收集的数据集，使用 d3rlpy 库的 CQL 算法进行离线训练。

```
1 from d3rlpy.algos import CQL
2 from d3rlpy.dataset import MDPDataset
3
4 dataset = MDPDataset.load('subway_dataset.h5')
5 cql = CQL(actor_encoder_factory='nature', discrete_action=True)
6 cql.fit(dataset, n_steps=200000)
7 cql.save_model('cql_subway.pt')
```

训练前需将 pickle 数据合并转换为 HDF5 格式。

7 测试与评估

项目提供了多个测试脚本确保各功能正常运行。

7.1 测试鼠标滑动

运行 `test_swipe.py`，依次执行四个方向的滑动，观察角色是否响应。

7.2 测试环境稳定性

运行 `test_env.py`，使用随机动作连续运行数千步，检查自动重启、奖励计算、信息提取是否正常。

7.3 测试防卡死

可通过手动模拟卡死（如暂停游戏）观察环境是否进入卡死模式并在超时后强制结束。

7.4 模型评估

训练好的模型可通过 `evaluate.py` 加载并在真实游戏中运行，记录平均奖励和步数。

8 未来工作

- 优化 OCR 速度和精度，减少环境交互延迟。

- 探索多环境并行训练，加速数据收集。
- 尝试更先进的离线算法（如 IQL、TD3+BC）。
- 将训练好的策略迁移到移动端或简化模拟环境中。